Plus d'entraînement = de meilleures performances ?

RQ3 : préentraînement spécialisé par langage

- Continuer l'entraînement avec du code Python ou Java

RQ4 : ajustement sur instructions

- Apprendre à suivre des demandes grâce à des exemples de réponses

Question commune : quel effet sur les tâches réelles de CoderUJB ?

Se spécialiser dans un langage : un compromis

StarCoder-15B

CoderUJB FCG / Java

Base	15,32 %
+ Python	14,39 %
+ Java	18,82 %

HumanEval / Python

Base	30,35 %
+ Python	33,57 %
+ Java	27,07 %

En FCG, la spécialisation favorise généralement le langage ciblé.

Mais ce résultat n'est pas systématique pour ITG et APR (RQ3).

L'instruction tuning n'améliore pas toutes les tâches

CodeLlama-13B

Tâche et indicateur	Base	Instruct
FCG : pass-all@1	21,91 %	13,28 %
CTG : pass-all@1	12,61 %	6,14 %
APR : pass-all@1	4,50 %	4,09 %
APR : pass-all@10	8,39 %	15,72 %

Tableau 5 : pourquoi la génération se dégrade-t-elle ?
FCG, CodeLlama-13B

Indicateur	Base	Instruct
Syntaxe	94,14 %	95,99 %
Compilation	63,49 %	40,76 %

La syntaxe reste correcte, mais le code compile moins souvent.

Plus d'entraînement $\neq$ meilleures performances

RQ3 : préentraînement par langage

La spécialisation aide certaines tâches, mais ses effets varient selon le langage et la tâche.

RQ4 : instruction fine-tuning

La génération peut se dégrader ; certaines réparations peuvent s'améliorer.

Interprétation des auteurs

La proximité entre préentraînement et tâche évaluée pourrait expliquer ces différences.

Cas particulier : CodeLlama-Instruct-34B produit des réponses répétées (mode collapse).